EF Core: Left Join

Concise lesson summary — optional related data, SQL translation, and highlighted key terms.

1. INNER JOIN vs LEFT JOIN

INNER JOIN returns only rows with matches on both sides. **LEFT JOIN** returns **all rows from the left source** plus matching right-side data. If there is no match, right-side columns are **NULL**.

Memory anchor: Need all main rows even when related data is missing? Think LEFT JOIN.

2. Example: Students with or without profiles

Students are the left source; StudentProfiles are the right source. Match them by StudentId. A student without a profile should still appear in the report.

```
var report =
    from s in context.Students
    join p in context.StudentProfiles
        on s.StudentId equals p.StudentId
        into profileGroup
    from p in profileGroup.DefaultIfEmpty()
    select new
    {
        s.StudentId,
        StudentName = s.FirstName + " " + s.LastName,
        City = p != null ? p.City : "No Profile",
        Country = p != null ? p.Country : "No Profile"
    };
```

3. LEFT JOIN LINQ pattern

```
join p in context.StudentProfiles
    on s.StudentId equals p.StudentId
    into profileGroup
from p in profileGroup.DefaultIfEmpty()
```

• **join ... into profileGroup** creates a grouped join.

• **DefaultIfEmpty()** lets a left-side row remain when there is no matching right-side row.

• **p != null ? p.City : "No Profile"** chooses a fallback value when no profile exists.

4. SQL generated in the lesson

```
SELECT s.StudentId,
       s.FirstName + N' ' + s.LastName AS StudentName,
       p.City, p.Country
FROM Students AS s
LEFT JOIN StudentProfiles AS p
    ON s.StudentId = p.StudentId
ORDER BY s.StudentId;
```

• **ToQueryString()** previews the SQL query shape; **ToList()** executes the query and materializes results.

5. Common mistakes

Mistake: using INNER JOIN for optional profile data. Students without profiles disappear.

Mistake: accessing p.City without checking for a missing profile. Handle the null case and choose an appropriate fallback.

6. Real use cases

• Students with or without profiles or enrollments.

• Instructors with or without managers.

• Products with or without discounts.

• Courses with or without enrollments.

7. Multiple LEFT JOINs

You can chain LEFT JOINs, such as Students → Profiles and Students → Enrollments, to preserve all students even when related records are missing.

Final takeaway

LEFT JOIN preserves all rows from the left source. Missing right-side data appears as NULL, so handle optional values safely.

Note: exact SQL depends on the LINQ query, model, provider, and EF Core version.